

SISTEMAS DISTRIBUÍDOS

Aula 18 — Algoritmos Distribuídos I: Consenso Distribuído

6º Semestre CC/ES-SI · Prof. Leandro Borges · 13/10

Agenda da Aula

- 1 O problema do consenso distribuído
- 2 Por que consenso é difícil: impossibilidade FLP
- 3 Paxos: o algoritmo clássico
- 4 Paxos vs. Raft: duas abordagens ao mesmo problema
- 5 Estudo de caso: consenso em sistemas reais

Consenso Distribuído: O Problema

O que é consenso

Em um sistema distribuído, múltiplos processos frequentemente precisam concordar sobre um único valor — quem é o coordenador atual, qual foi a última operação confirmada, qual réplica contém os dados corretos — mesmo quando mensagens se perdem, atrasam ou alguns processos falham.

Propriedades exigidas de um algoritmo de consenso

Acordo — todos os processos corretos decidem o mesmo valor. Validade — o valor decidido foi realmente proposto por algum processo. Terminação — todo processo correto eventualmente decide um valor, sem travar para sempre.

Por Que Consenso é Difícil

O resultado FLP (1985)

Fischer, Lynch e Paterson provaram que, em um sistema puramente assíncrono, nenhum algoritmo determinístico garante consenso na presença de sequer uma falha de processo — não há como distinguir com certeza um processo lento de um processo que falhou.

Contornando a impossibilidade na prática

Sistemas reais escapam da barreira teórica assumindo sincronia parcial (timeouts), introduzindo aleatoriedade nas decisões, ou aceitando terminação apenas "com alta probabilidade" em vez de garantia absoluta — é exatamente isso que Paxos e Raft fazem.

Paxos: O Algoritmo Clássico

Fase 1 — Prepare / Promise

Um processo proponente escolhe um número de proposta e envia Prepare a uma maioria (quórum) de aceitadores. Cada aceitador que ainda não viu proposta mais recente responde com Promise, comprometendo-se a ignorar propostas mais antigas.

Fase 2 — Accept / Accepted

Se o proponente recebe Promise de uma maioria, envia Accept com o valor. Cada aceitador aceita, a menos que já tenha prometido a uma proposta mais nova. Quando uma maioria aceita, o valor está decidido — de forma irreversível.

Paxos vs. Raft: Duas Abordagens ao Mesmo Problema

Aspecto	Paxos	Raft
Papel dos nós	Simétrico: qualquer nó pode propor	Assimétrico: um líder eleito coordena
Como decide	Quóruns de Prepare/Accept por rodada	Líder replica entradas de log aos seguidores
Papel do líder	Não formaliza um líder explícito	Eleição de líder é parte central do protocolo
Facilidade de entendimento	Historicamente difícil de implementar corretamente	Projetado explicitamente para ser compreensível

Estudo de Caso: Consenso em Sistemas Reais

etcd

Armazenamento chave-valor distribuído usado pelo Kubernetes para guardar o estado do cluster. Usa Raft para replicar seu log entre réplicas e eleger um líder — se o líder cai, um novo é eleito automaticamente em segundos.

Apache ZooKeeper

Serviço de coordenação usado por Kafka, Hadoop e outros sistemas distribuídos. Usa o protocolo ZAB (ZooKeeper Atomic Broadcast), inspirado nas ideias do Paxos, para manter todas as réplicas de configuração consistentes.

Próxima Aula

Algoritmos Distribuídos II: Snapshot Distribuído e Detecção de Terminação

20/10 — como capturar o estado global de um sistema em execução.

Referências desta Aula

- LAMPORT, L. The Part-Time Parliament. ACM Transactions on Computer Systems, v. 16, n. 2, 1998.
- ONGARO, D.; OUSTERHOUT, J. In Search of an Understandable Consensus Algorithm (Raft). USENIX Annual Technical Conference, 2014.
- TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.